

Change Report

Group Number: Cohort 1, Group 11

Group Name: Y111 Studios

Group Members:

1. James Hutchinson
2. Somto Igweonu
3. Robert Kisloski
4. Sam Knight
5. Kenneth Kok
6. Ashish Kumar

This document outlines the procedure that we followed to make and keep track of changes to the Assessment 1 deliverables, documentation and code that we inherited from Team 9.

Planning the Changes

We started by carrying out a cross-team review of the current system and the accompanying deliverables to get a feel for the changes that we would want to make and the impact that they would have on the product.

We reviewed the existing code and documentation to evaluate dependencies, and we have prioritised features and bugs to ensure that the changes do not break the functionality of the game. We have used the notes from our client interview and the renewed product brief to identify any changes that we need to make to the user and system requirements, and these updated requirements have allowed us to identify any changes that we need to make to the product architecture. Finally, we have reflected on our work from Assessment 1 to identify any changes that we need to make to the risk assessment and the method selection and planning deliverables.

Making the Changes

Similar to Assessment 1, we have made notes of the changes that we would like to make on shared Google Docs files and by hand-writing them. This allows us to discuss our ideas together and makes it easier to make the final decisions, while still allowing individual group members to maintain a personalised track of the changes. This also allows us to make rough sketches of the changes that we need to make to the deliverables before compiling all the changes into the final documents.

Again similar to Assessment 1, the code changes have been tracked through Git, with clear commit messages to provide context for which issues they address. We have continued to use GitHub Issues to document and manage tasks, making sure that each issue is clearly defined with the scope of the change, and that they reference relevant parts of the code affected. The changes that we make are in separate branches to avoid conflict with other ongoing work, and the branches are typically named according to the issue or feature that they address.

Reviewing the Changes

The shared Google Docs files allow all group members to review each other's work and provide feedback, thus allowing us to remove any redundancies and decide on what changes are the best to make for each of the deliverables.

We have continued with our established peer review process, where we use Pull Requests (PRs) for reviewing and discussing code changes before merging them into the main branch. We have used GitHub Actions to automate build, test, and deployment pipelines to ensure that code changes do not break functionality and that all tests pass before merging.

Requirements Changes

Original Deliverable: <https://y111studios.github.io/unisim2/deliverables/Req1.pdf>

Updated Deliverable: <https://y111studios.github.io/unisim2/deliverables/Req2.pdf>

As the team takes on the project of a different team, some requirements have been updated to better suit the intended player-base of the game and to improve the gaming experience. The requirement changes fall into two categories; user requirements and functional requirements.

Changes to User Requirements

In the user requirements, some priorities were elevated to more accurately reflect the needs and objectives for assessment 2. The events and student satisfaction mechanics, which were not prioritised in assessment 1, were given a higher importance as a result.

In addition, four new requirements were introduced: the score counter, leaderboard, and achievements and camera control. The score counter, leaderboard and achievements were added as they are the new requirements that have been identified through the updated product brief. While camera control is not critical to the success of Assessment 2, the team chose to include it due to the lack of a meaningful way of camera manipulation by the player.

Changes to Functional Requirements

In functional requirements, only the building requirement description was updated to better specify the actions that the system must perform. Four new requirements were also added: score, leaderboard, achievement and camera control. These functional requirements are tied to their user requirements counterparts, as the new mechanics are essential for this phase of the assessment.

Constraint Requirements

We have decided not to make any changes to the constraint requirements, as the ones identified by the previous team sufficiently describe these requirements for our project.

Architecture Changes

Original Deliverable: <https://y111studios.github.io/unisim2/architecture.html>

Updated Deliverable: <https://y111studios.github.io/unisim2/deliverables/Arch2.pdf>

We have implemented significant changes to the architecture by adding over 30 new classes to the existing code. Additionally, we have modified the existing classes, making changes such as adding or removing class variables. These changes were essential to accommodate the new features required in Assessment 2 for this phase of the project.

Achievements

The achievements required a new achievement object to be made in order to make it easy to add new ones, and a manager class so that these achievements could be more easily stored and given to the player. An enum class was created to store all the achievements in the game, and another achievement tracker class was made too so that data could be collected from the player to determine if they need to be awarded an achievement. This would not have been possible without adding a completely new package with classes dedicated to this.

User Interface

Ten new classes were added to the existing UI package in order to add new UI elements that were needed to display information from new features that had been added, as well as for quality of life improvements. Most of these new classes are just new states for the game to switch to when a button or key is pressed, for example one class is a pause screen state, which allows the user to return to the main menu or resume the game. This state contains variables such as a TextButton that the user can click to choose what to do. The classes in the UI package mostly contain the same methods, as they use the built in LibGDX graphics support.

Scoring

Five new classes were added to allow for the player's score to be calculated. A satisfaction tracker was made to track the player's current user satisfaction, a satisfaction modifier class was created to call functions from other classes to modify the current student satisfaction, a score tracker was made, a score object that has a float and an update interval and finally a satisfaction need class to calculate the current satisfaction.

Building

Only one new class was added to the building package, but the two existing classes from assessment 1 were heavily modified in order to complete the requirements for Assessment 2. The new class added, BuildingType, is an enum that specifies four different building types which are Recreation, Learning, Sleeping and Eating. These types were added to allow for them to be used by other classes that could need this data. BuildingManager had 9 new methods added. These include getBuildingCapacities, which returns the capacity of all the buildings apart from the currently selected one, getBuildings which returns all buildings on the map, drawBuilding which draws a building at the given vector location, getBuildingCount which returns the number of buildings, decrementCounters which reduces the number of buildings stored in buildingCounts, and updateCounters which increments the building counts.

Events

Events was a new package added to allow the addition of events to the game and it contains three new classes. These include ChoiceCard, which is used to represent a choice during an event, EventCard which can contain a ChoiceCard and is used to show an event to the user and EventBucket which holds all the events and is used to randomly select an event for the user.

Finance

The new finance package only contains one class called MoneyTracker, and this is used to keep track of the player's money.

World

The world package is not new to the project, but some of its classes contain new variables which are objects of some of the other new classes added, such as moneyTracker which is now a variable of the World class.

Leaderboard

The leaderboard package was added to allow for the leaderboard requirement to be satisfied and it contains two classes, Leaderboard and LeaderboardEntry. The Leaderboard class creates a json file on the player's computer and stores the inputted name and score in it and the LeaderboardEntry class creates a new entry to the leaderboard based on the player's name and score.

Other Changes

Small changes and bug fixes were made to other classes, such as adding new gameplay states to the **GameState** class, fixing a minor bug in the **Timer** class, and fixing a bug in the **Point** class.

Method Selection & Planning Changes

Original Deliverable: <https://y111studios.github.io/unisim2/deliverables/Plan1.pdf>

Updated Deliverable: <https://y111studios.github.io/unisim2/deliverables/Plan2.pdf>

As we are a different group who have different preferences and ideas, considerable changes have been made to the method selection and planning for this phase of the project.

Changes to Engineering Methods

The previous team chose to use an agile software engineering methodology for more flexibility, whereas the main software engineering method for our team was feature-driven development. We decided to make this change as this promotes quick results and emphasises good object-oriented design.

For the other engineering methods, we decided not to make too many changes. We continued to use GitHub for version control, LibGDX as the development framework for our game, and Discord as the tool for communication, as these were tools with which we are already familiar. We continued to use Gradle as the project management system since it continued to provide a quick and easy way to build a LibGDX project.

Changes to Team Organisation

The previous team had a defined team organisation where the members had specific roles. We did not continue with this approach, as we did not have any specific responsibilities for any of the team members and we did not assign any single project leader. Instead, we decided to evenly split the work together to fit our strengths and proceeded with a system of peer reviewing and team discussions. We believe that this change allowed for greater creativity and allowed for multiple parts of the project to progress rapidly at the same time.

The Project Plan

Our project plan had a structure similar to the previous team, with changes being made throughout the weeks based on our progress.

Risk Assessment & Mitigation Changes

Original Deliverable: <https://y111studios.github.io/unisim2/deliverables/Risk1.pdf>

Updated Deliverable: <https://y111studios.github.io/unisim2/deliverables/Risk2.pdf>

We have decided not to make any changes to the Risk Assessment carried out by the previous team, as we believe that their assessment is comprehensive enough to cover any risks which we could run into during the project.

We have also decided to continue with the same structure for the table presenting the risk register, since this structure covers all the necessary aspects of the risk assessment process, clearly describing each of the risks and highlighting their likelihood and impact. The only changes that we have made are to the ownership of each of the risks to update them to the members of our group.